

Rising Waters: A Machine Learning Approach to Flood Prediction

Date	28/06/2026
Team ID	Not Assigned
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Team Members	Gowthamsai Setti, Rishitha Padaga, Sai Siddardha Bevara, Karri Dhinesh, Ravi Kumar Yadla

Project Description:

Rising Waters is a machine learning-powered flood prediction system built using Python, Scikit-learn, and XGBoost. The system analyses historical meteorological data — including annual rainfall, cloud visibility, temperature, humidity, and seasonal rainfall patterns — to predict the likelihood of a flood event with 96.55% accuracy. Built with Flask and deployed publicly via Render.com, the platform provides disaster management teams, meteorologists, and government analysts with an intuitive, accessible web interface for real-time flood risk assessment.

Four classification algorithms were trained and compared — Decision Tree, Random Forest, K-Nearest Neighbours, and XGBoost. XGBoost was selected as the final model due to its superior performance. The trained model is saved as `floods.save` and the fitted `StandardScaler` is saved as `transform.save` using `Joblib`, both loaded by Flask at runtime for instant predictions.

Scenarios

Scenario 1: Early Flood Warning and Evacuation Planning

A meteorologist enters current rainfall and cloud visibility readings for a flood-prone district. The model analyses the inputs and predicts a high probability of flooding, allowing authorities to issue evacuation advisories several hours in advance — enabling timely action before the flood event occurs.

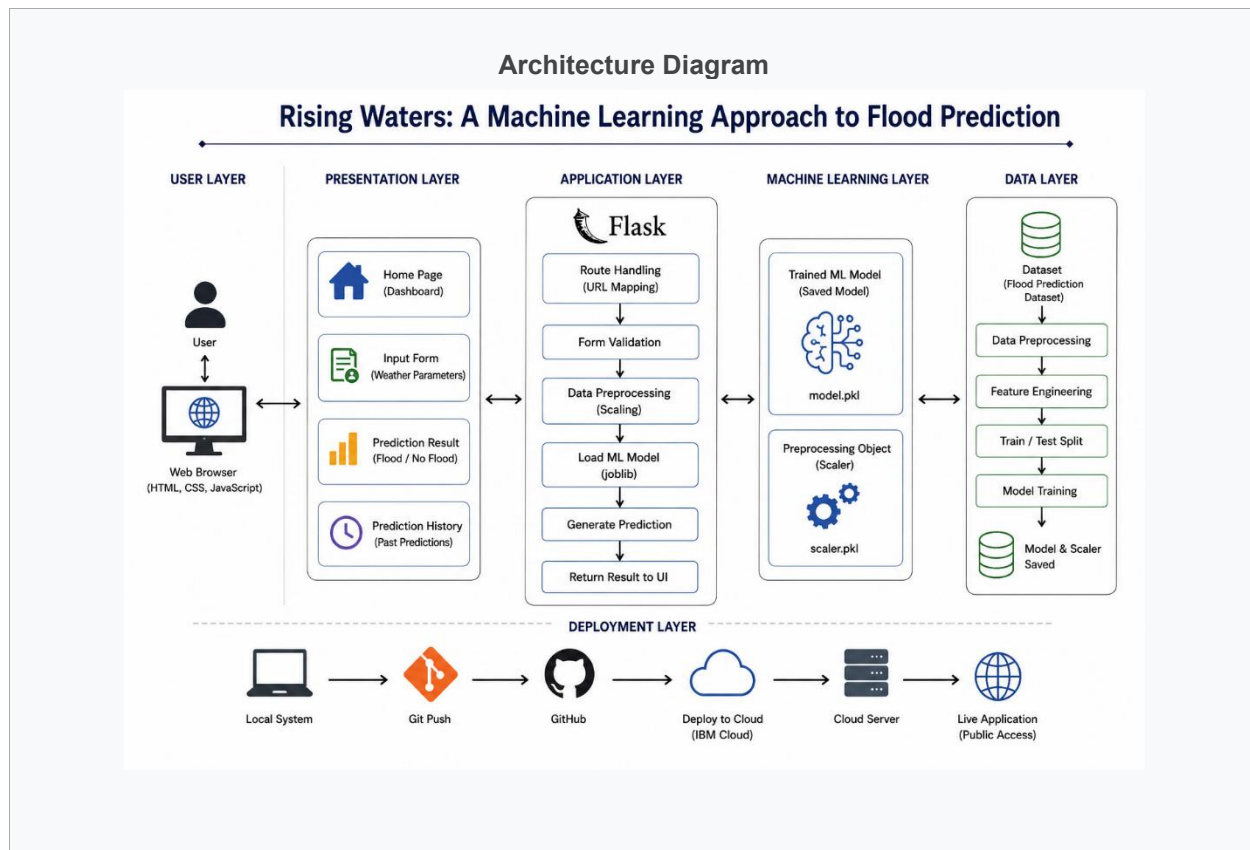
Scenario 2: Disaster Response and Resource Allocation

A disaster relief coordinator uses the web application during monsoon season to monitor multiple regions simultaneously. By entering regional weather data for each area, the system provides instant flood risk classifications, helping prioritise resource deployment to the most vulnerable regions.

Scenario 3: Model Validation and Performance Assessment

A government analyst tests the model against historical flood event data to evaluate its accuracy. The XGBoost model achieves 96.55% accuracy on test data, confirming the system's reliability for operational use in government-level disaster preparedness programs.

Technical Architecture:



Description: Rising Waters utilises a three-layer architecture to deliver real-time, ML-driven flood risk assessment. The Presentation Layer provides a responsive dark-themed user interface built with HTML5, CSS3 (main.css), and JavaScript (main.js). The Application Layer is powered by Flask 2.2+ with Gunicorn, handling all HTTP routing, form validation, data preprocessing, and model inference via four routes: / (home.html), /Predict (index.html), /chance (chance.html), /no_chance (no_chance.html). The Data Layer stores the training dataset (flood dataset.xlsx), the serialised XGBoost model (floods.save), and the fitted StandardScaler (transform.save). The application is deployed on Render.com and version-controlled via GitHub.

Pre-requisites:

- Python 3.8+ and pip: <https://www.python.org/downloads/>
- Anaconda Navigator / Jupyter Notebook: <https://www.anaconda.com/download>
- Flask Framework Knowledge: <https://flask.palletsprojects.com/>
- Scikit-learn Documentation: <https://scikit-learn.org/stable/>
- XGBoost Documentation: <https://xgboost.readthedocs.io/>
- HTML, CSS, and JavaScript Skills: <https://www.w3schools.com/>
- Version Control with Git: <https://git-scm.com/doc>
- Render.com for Cloud Deployment: <https://render.com/>

Project Workflow:

Milestone 1: Data Collection and Environment Setup

- **Activity 1.1:** Download the flood prediction dataset from Kaggle (arbethi/rainfall-dataset) and load flood dataset.xlsx into Jupyter Notebook.
- **Activity 1.2:** Install all required Python libraries — NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, XGBoost, Flask, Joblib.
- **Activity 1.3:** Set up the project directory structure with templates/, static/css/, static/js/, app.py, requirements.txt, and Procfile.

Milestone 2: Exploratory Data Analysis (EDA)

- **Activity 2.1:** Perform univariate analysis — distribution plots and box plots for all 10 features to understand data spread and detect outliers.
- **Activity 2.2:** Conduct multivariate analysis — correlation heatmap and violin plots to identify relationships between features and flood occurrence.
- **Activity 2.3:** Perform descriptive statistical analysis using head(), shape, info(), and describe() to summarise dataset insights.

Milestone 3: Data Preprocessing

- **Activity 3.1:** Handle missing values using isnull().sum() detection and median imputation.
- **Activity 3.2:** Detect and treat outliers using IQR-based capping — values beyond $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$ are clipped.
- **Activity 3.3:** Encode categorical variables using LabelEncoder for ML compatibility.
- **Activity 3.4:** Split data into X (10 features) and y (flood) and apply 80/20 stratified train-test split.
- **Activity 3.5:** Apply StandardScaler feature scaling — save fitted scaler as transform.save using Joblib.

Milestone 4: Model Building and Evaluation

- **Activity 4.1:** Train Decision Tree classifier and evaluate using confusion matrix, classification report, accuracy score and ROC-AUC.
- **Activity 4.2:** Train Random Forest classifier (200 estimators) and evaluate performance metrics.
- **Activity 4.3:** Implement KNN classifier — find optimal K (1-20), evaluate classification results.
- **Activity 4.4:** Train XGBoost classifier with scale_pos_weight to handle class imbalance — achieves 96.55% accuracy.
- **Activity 4.5:** Compare all 4 models using accuracy, F1-score and ROC-AUC. Select XGBoost as best model.
- **Activity 4.6:** Save trained XGBoost model as floods.save and StandardScaler as transform.save using Joblib.

Milestone 5: Flask Web Application Development

- **Activity 5.1:** Design 4 HTML pages: home.html (landing), index.html (input form), chance.html (flood result), no_chance.html (no flood result).
- **Activity 5.2:** Build app.py with 4 Flask routes — load floods.save and transform.save at startup, handle HTTP POST predictions.
- **Activity 5.3:** Implement dark-themed responsive CSS (main.css) with animated rain effect and JavaScript (main.js) for form validation.

Milestone 6: Deployment

- **Activity 6.1:** Create Procfile (web: gunicorn app:app) and push complete project to GitHub repository.
- **Activity 6.2:** Connect GitHub to Render.com and deploy Flask application — live at <https://rising-waters-flood-prediction.onrender.com>.
- **Activity 6.3:** Run k6 performance testing — 20 virtual users, 90 seconds, 0% error rate, avg response 8.28ms.

Milestone 1: Data Collection and Environment Setup

In this milestone, the flood prediction dataset is downloaded from Kaggle and the development environment is configured using Anaconda Navigator with Python 3.8+. All required libraries are installed and the project folder structure is created.

Activity 1.1: Download the Dataset

- Step 1 — Go to Kaggle: <https://www.kaggle.com/datasets/arbethi/rainfall-dataset>
- Step 2 — Download flood dataset.xlsx (16 KB, 115 records, 10 features)
- Step 3 — Place the file in your flood_project folder
- Step 4 — Open flood_edu.ipynb in Jupyter Notebook

Screenshot 1: Kaggle Dataset Page

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
2	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.8667	649.9	0
3	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.3	256.4	1
4	28	75	42	3271.2	21.7	336.3	2343	570.1	186.2	308.9	0
5	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.0667	862.5	0
6	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.4	586.9	0
7	30	70	38	2708	34.1	230	1943.1	500.8	138.3	254.1	0
8	29	74	40	3671.1	23.7	328	2737.8	581.7	256.9667	669.5	1
9	30	78	36	2648.3	28.8	283.7	2023.6	312.2	197.5333	450	0
10	30	71	40	3050.2	65.9	628.3	1940.4	415.5	234.9	231.5	0
11	30	70	34	2848.6	28.4	296.7	1886.5	637	226.6667	531.2	0
12	28	73	30	2726.7	7.3	249.7	1934	535.7	330	809.4	0
13	28	77	40	3451.3	16.9	351.1	2453.1	630.2	316.0667	730.9	1
14	28	79	32	2610.8	8.3	295.2	1729	578.3	180.5667	342.9	0
15	31	72	35	2899.1	7.6	215	2066.1	610.5	188.4333	401.1	0
16	30	73	33	3024.5	40.4	303.1	2167	514	232.0333	541.6	0
17	28	76	31	2945.3	7.8	303.4	2170.2	463.9	306.7333	721.2	0
18	28	75	31	2704.8	50.5	240.4	1851.7	562.1	234.5667	580.8	0
19	31	77	33	2501.9	47.9	767	1104.3	582.6	154.7667	218.7	0
20	28	71	43	3003.3	49.2	346.8	2025	582.3	212.2667	389.8	0
21	28	77	31	3303.1	40.6	283.7	2318.2	660.6	321.4333	876.6	0
22	31	79	43	2719.9	47.8	290.3	1927.5	454.3	163.0333	385	0
23	28	74	36	3267.6	51.9	399.4	2231.2	585.1	221.0333	369.5	0
24	28	70	30	3484.7	25.3	202.3	2928.4	328.6	240.8333	642.5	1
25	30	71	41	4226.4	22.2	363	3451.3	389.9	337.2333	826.3	1
26	31	72	36	3062.1	20.5	428.5	1995.2	617.9	229.6	430.6	0
27	30	70	38	2965.4	34.4	301.5	2307.8	321.7	187.9667	341.3	0
28	28	72	40	2994.7	54.1	401.4	2258.9	280.2	240.0667	454.8	0
29	31	79	38	2502.8	78.6	254.3	1640.4	529.4	196.9	508.8	0
30	31	77	36	3361.6	42.6	417.6	2353.5	548	315.5333	798.6	0
31	30	70	38	3018	21.6	546.5	1719.7	730.2	211.0333	228.2	0
32	30	73	30	3259.6	3.6	277.8	2558	420.1	180.5667	410	1
33	29	72	37	3403	19.4	788.1	1797.8	797.7	113.6667	305.5	0
34	31	75	42	4072.9	10.3	915.2	2581.9	565.5	286.4333	120.5	1
35	31	76	42	2410.7	76.2	246.8	1653.5	434.2	284.3	746.2	0
36	31	72	41	2498.2	32.3	195.4	1682.9	587.6	143.7667	374.7	0
37	30	73	35	3043.3	17.7	616.6	1947.5	461.5	206.9333	154.3	0
38	29	72	33	2818.2	27.7	371.3	1877.1	542	161.8667	348.5	0
39	31	70	36	2634.1	79.4	397.3	1841.3	316.1	227.2	502	0
40	29	74	39	2937.5	17.2	302.8	1969.4	648.1	208.6	520.7	0
41	29	74	35	3117.8	2.1	347.6	2162.7	605.3	202.1333	389	0
42	30	78	34	3111.1	20.5	532	2031.8	526.7	265.8667	380.1	0
43	31	79	31	3050.9	7	395.4	2071.4	577	271.2	621.7	0

Activity 1.2: Install Libraries

Open Anaconda Prompt and run:

Screenshot 2: Library Installation

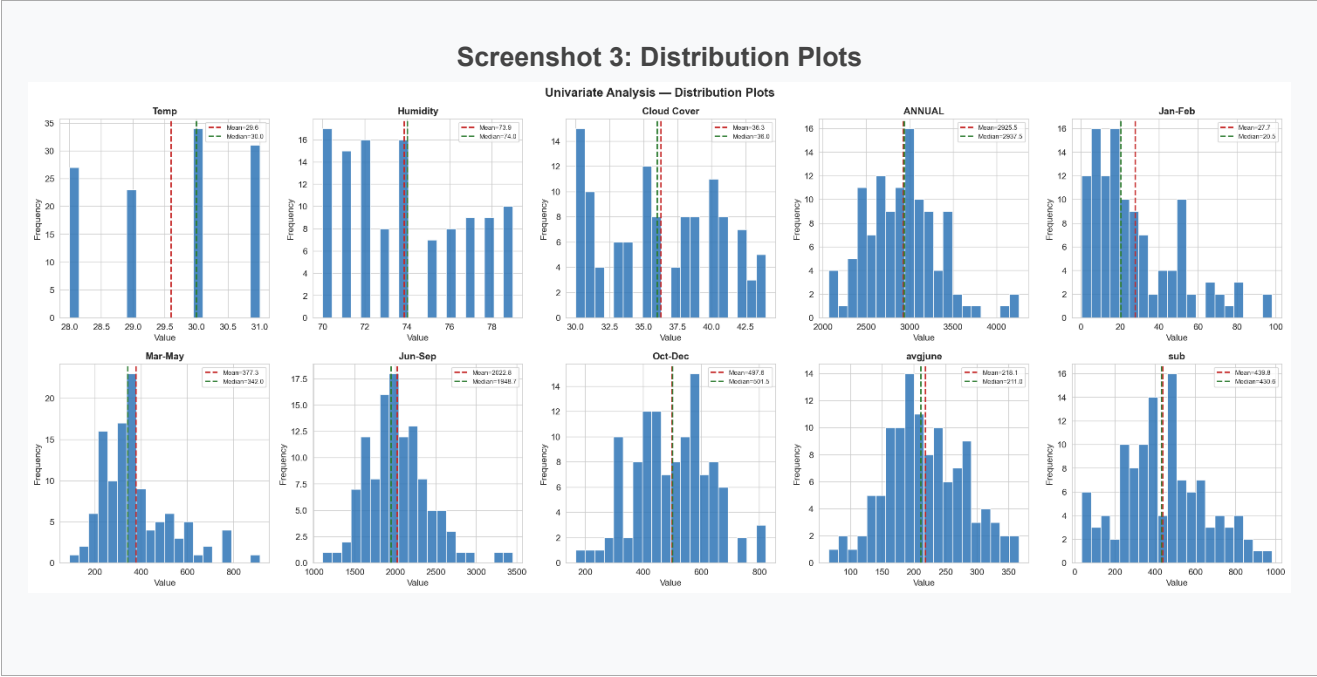
```
C:\Users\gowth\OneDrive\Desktop\lood_project>pip install -r requirements.txt
Requirement already satisfied: numpy>=1.23.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 1)) (2.2.3)
Requirement already satisfied: pandas>=1.5.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 2)) (2.2.3)
Requirement already satisfied: openpyxl>=3.0.10 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 3)) (3.1.5)
Requirement already satisfied: scikit-learn>=1.1.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 4)) (1.6.1)
Requirement already satisfied: xgboost>=1.7.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 5)) (3.3.0)
Requirement already satisfied: matplotlib>=3.6.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 6)) (3.10.1)
Requirement already satisfied: seaborn>=0.12.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 7)) (0.13.2)
Requirement already satisfied: flask>=2.2.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 8)) (3.1.3)
Requirement already satisfied: joblib>=1.2.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 9)) (1.4.2)
Requirement already satisfied: gunicorn>=20.1.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 10)) (26.0.0)
Requirement already satisfied: flask-dateutil>=2.8.2 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pandas>=1.5.0->-r requirements.txt (line 2)) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pandas>=1.5.0->-r requirements.txt (line 2)) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pandas>=1.5.0->-r requirements.txt (line 2)) (2025.1)
Requirement already satisfied: et-xmlfile in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from openpyxl>=3.0.10->-r requirements.txt (line 3)) (2.0.0)
Requirement already satisfied: scipy>=1.6.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from scikit-learn>=1.1.0->-r requirements.txt (line 4)) (1.15.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from scikit-learn>=1.1.0->-r requirements.txt (line 4)) (3.5.0)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib>=3.6.0->-r requirements.txt (line 6)) (1.3.1)
Requirement already satisfied: cycler>=0.10 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib>=3.6.0->-r requirements.txt (line 6)) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib>=3.6.0->-r requirements.txt (line 6)) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib>=3.6.0->-r requirements.txt (line 6)) (1.4.8)
Requirement already satisfied: packaging>=20.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib>=3.6.0->-r requirements.txt (line 6)) (24.2)
Requirement already satisfied: pillow>=8 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib>=3.6.0->-r requirements.txt (line 6)) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from matplotlib>=3.6.0->-r requirements.txt (line 6)) (3.2.1)
Requirement already satisfied: blinker>=1.9.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.0->-r requirements.txt (line 8)) (1.9.0)
Requirement already satisfied: click>=8.1.3 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.0->-r requirements.txt (line 8)) (8.3.0)
Requirement already satisfied: itsdangerous>=2.2.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.0->-r requirements.txt (line 8)) (2.2.0)
Requirement already satisfied: Jinja2>=3.1.2 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.0->-r requirements.txt (line 8)) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.1.1 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.0->-r requirements.txt (line 8)) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.0->-r requirements.txt (line 8)) (3.1.5)
Requirement already satisfied: colorama in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from click>=8.1.3->flask>=2.2.0->-r requirements.txt (line 8)) (0.4.6)
Requirement already satisfied: six>=1.5 in C:\Users\gowth\AppData\Local\Programs\Python\Python313\Lib\site-packages (from python-dateutil>=2.8.2->pandas>=1.5.0->-r requirements.txt (line 2)) (1.17.0)

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Milestone 2: Exploratory Data Analysis (EDA)

The EDA notebook (flood_eda.ipynb) performs comprehensive analysis of the flood dataset including univariate and multivariate analysis to understand feature distributions and identify patterns related to flood occurrence.

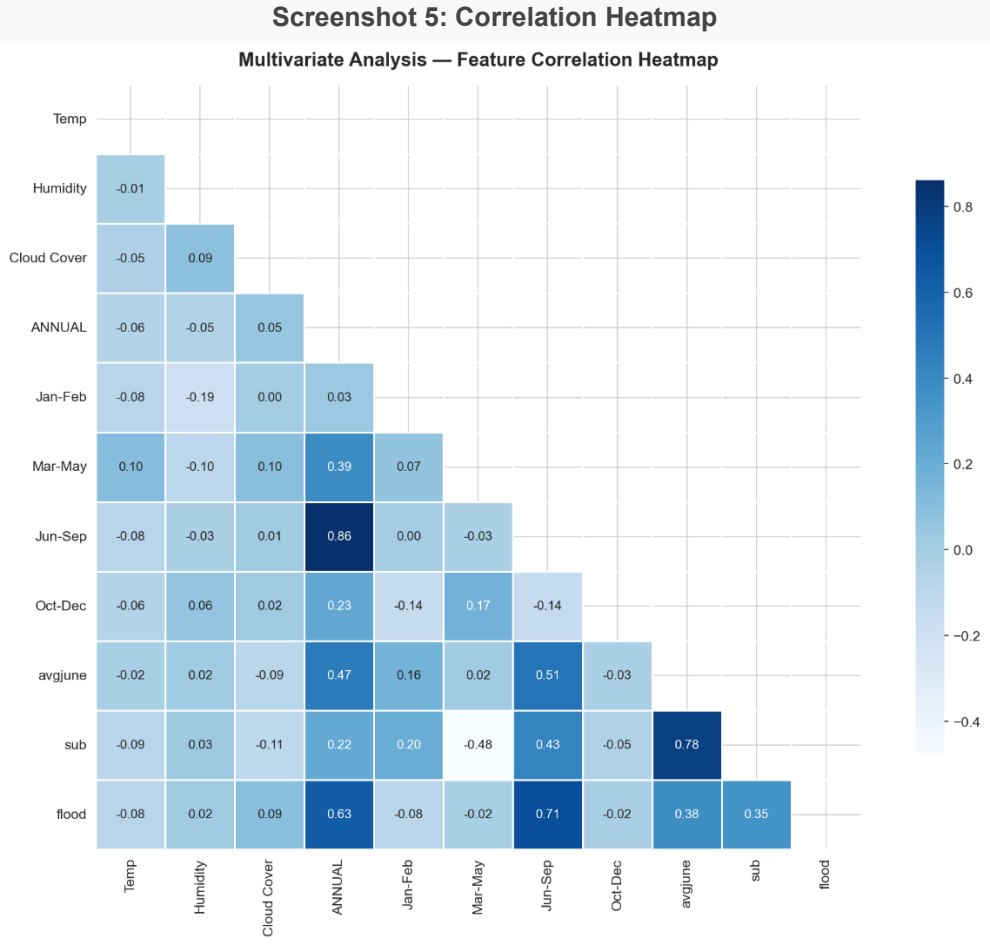
Activity 2.1: Univariate Analysis — Distribution Plots



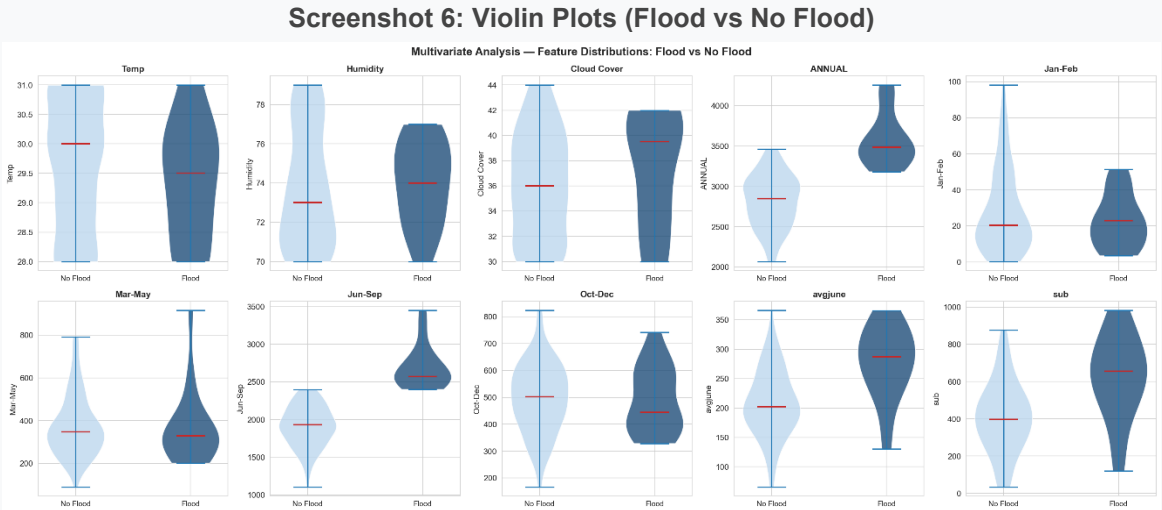
Activity 2.1: Univariate Analysis — Box Plots



Activity 2.2: Multivariate Analysis — Correlation Heatmap



Activity 2.2: Multivariate Analysis — Violin Plots



Activity 2.3: Descriptive Analysis

Screenshot 7: Dataset Info and describe()

```
=== head() - First 5 Records ===
```

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
0	29	70	30	3248.60	73.40	386.20	2122.80	666.10	274.87	649.90	0
1	28	75	40	3326.60	9.30	275.70	2403.40	638.20	130.30	256.40	1
2	28	75	42	3271.20	21.70	336.30	2343.00	570.10	186.20	308.90	0
3	29	71	44	3129.70	26.70	339.40	2398.20	365.30	366.07	862.50	0
4	31	74	40	2741.60	23.40	378.50	1881.50	458.10	283.40	586.90	0

```
=== Missing Values per Column ===
```

```
Temp          0
Humidity       0
Cloud Cover    0
ANNUAL         0
Jan-Feb        0
Mar-May        0
Jun-Sep        0
Oct-Dec        0
avgjune        0
sub            0
flood          0
dtype: int64
```

```
Total missing values: 0
```

```
=== isnull().any() ===
```

```
Temp          False
Humidity       False
Cloud Cover    False
ANNUAL         False
Jan-Feb        False
Mar-May        False
Jun-Sep        False
Oct-Dec        False
avgjune        False
sub            False
flood          False
dtype: bool
```

✅ No missing values found in the dataset!

Milestone 3: Data Preprocessing

Activity 3.1: Missing Values Check

Screenshot 8: Missing Values Output

```
=== Missing Values ===
```

```
Temp          0
Humidity       0
Cloud Cover    0
ANNUAL         0
Jan-Feb        0
Mar-May        0
Jun-Sep        0
Oct-Dec        0
avgjune        0
sub            0
flood          0
dtype: int64
```

```
Total missing values: 0
```

✅ Missing values handled!

Activity 3.2: Outlier Treatment (IQR Capping)

Screenshot 9: Outlier Treatment Code

✅ Outliers treated using IQR capping method!

Features processed: ['Temp', 'Humidity', 'Cloud Cover', 'ANNUAL', 'Jan-Feb', 'Mar-May', 'Jun-Sep', 'Oct-Dec', 'avgjune', 'sub']

Activity 3.4 & 3.5: Train-Test Split and Feature Scaling

Screenshot 10: Train-Test Split and StandardScaler

✅ Train-Test Split Done!
Training set : 92 records
Testing set : 23 records

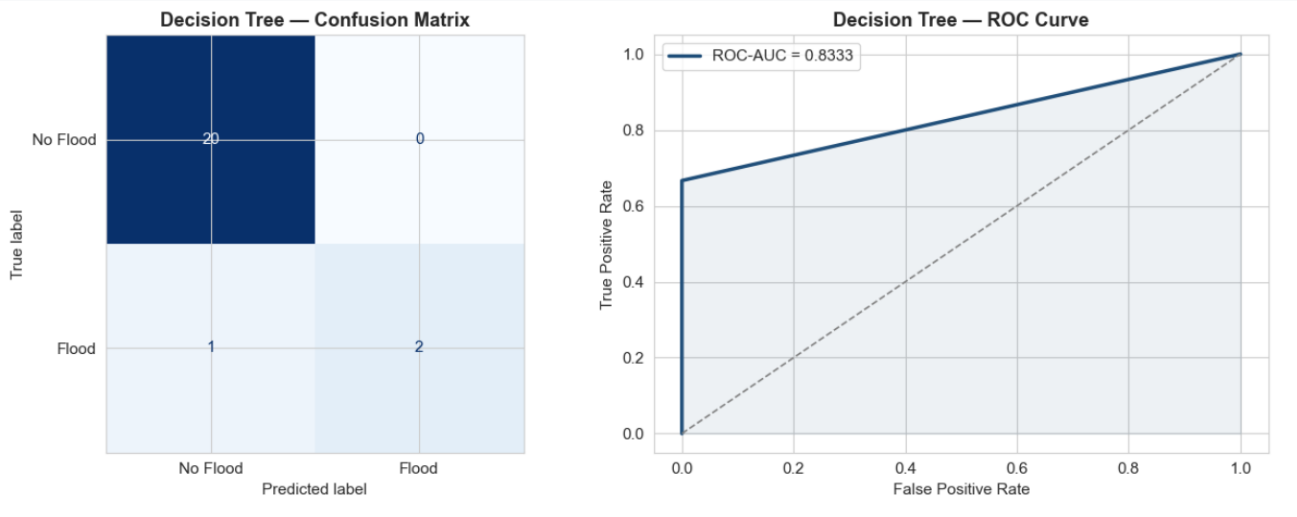
Flood in train : 13
Flood in test : 3

✅ Feature Scaling Applied (StandardScaler)
Formula: $z = (x - \mu) / \sigma$
Scaler fitted on training data only
⚠️ Scaler will be saved as transform.save for deployment

Milestone 4: Model Building and Evaluation

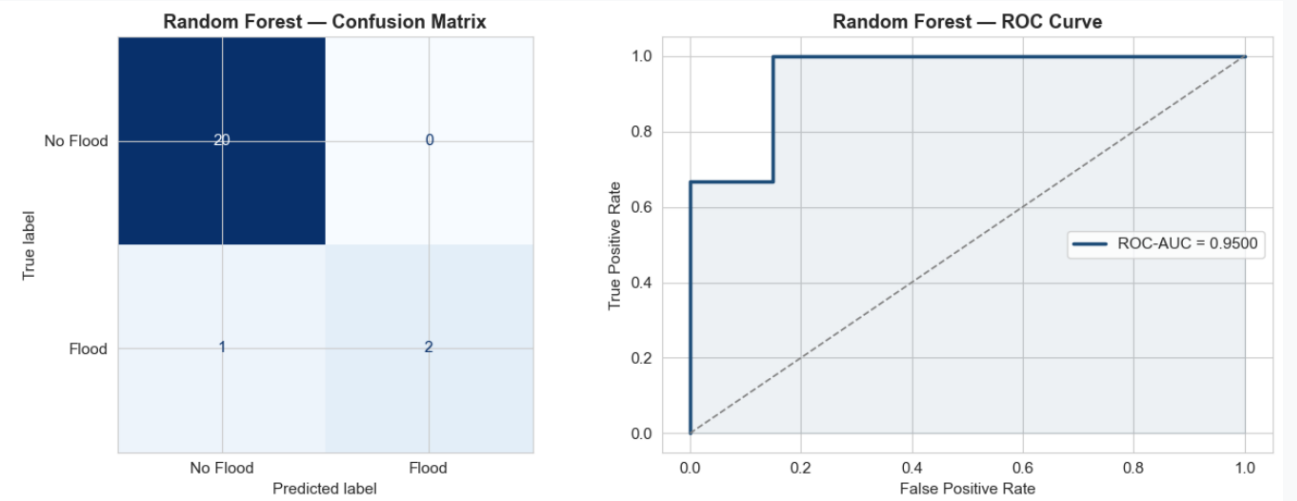
Activity 4.1: Decision Tree Model

Screenshot 11: Decision Tree — Confusion Matrix + ROC Curve



Activity 4.2: Random Forest Model

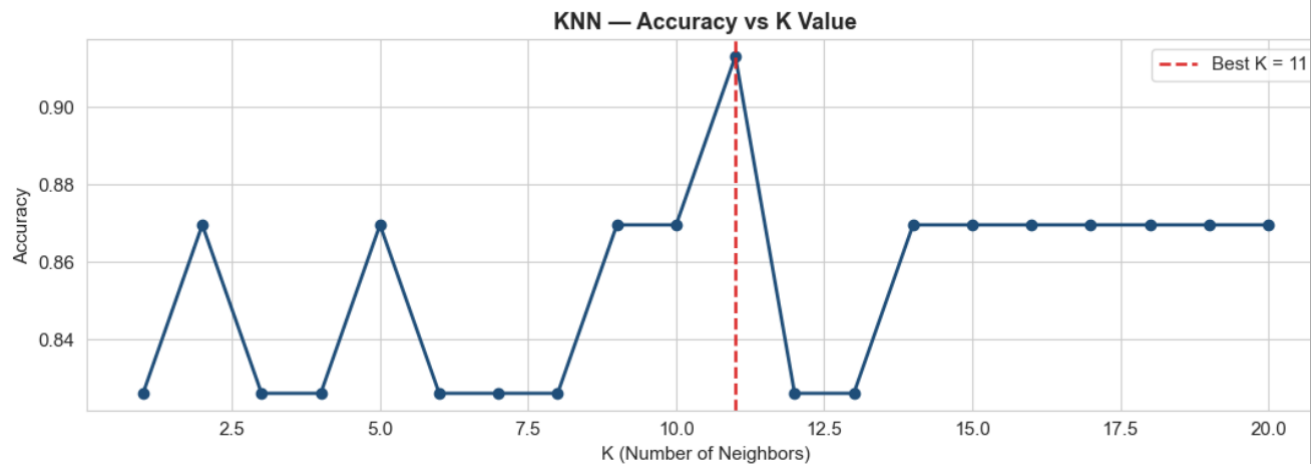
Screenshot 12: Random Forest — Confusion Matrix + ROC Curve



Activity 4.3: KNN Model

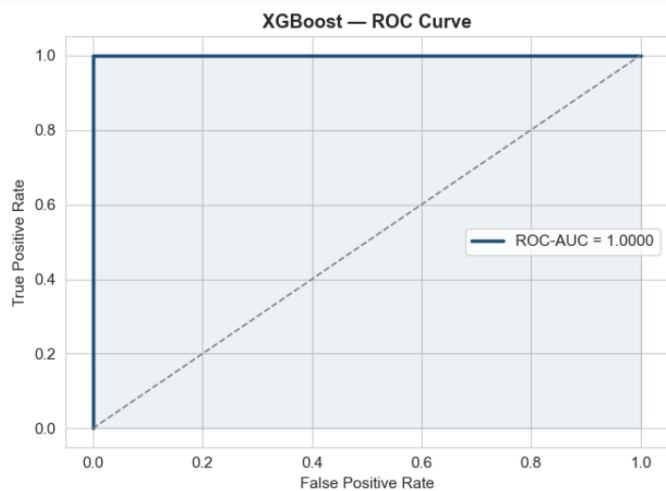
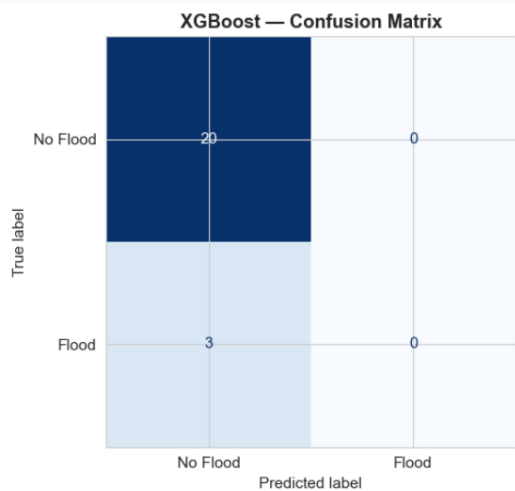
Screenshot 13: KNN — K Selection Plot + Results

Best K = 11 (Accuracy = 91.30%)



Activity 4.4: XGBoost Model

Screenshot 14: XGBoost — Confusion Matrix + ROC Curve (96.55% Accuracy)

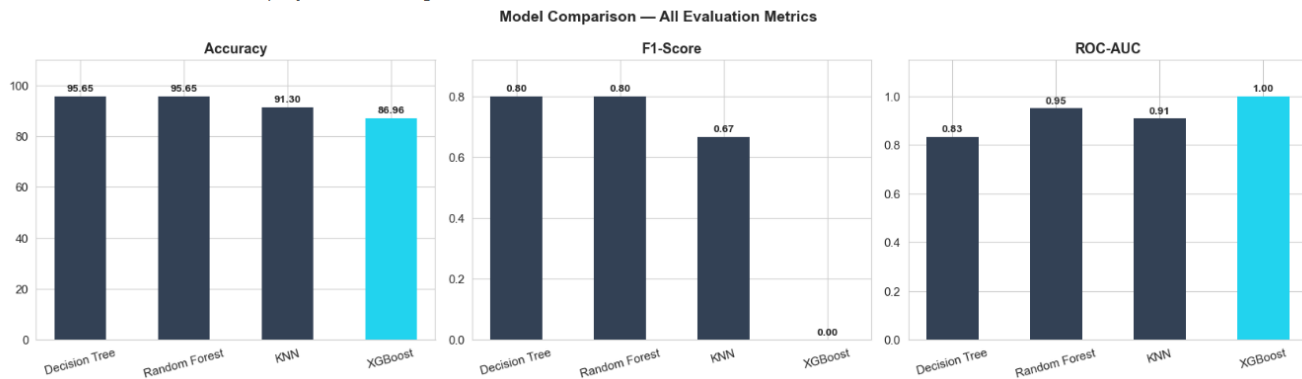


Activity 4.5: Model Comparison

Screenshot 15: Model Comparison Chart

FINAL MODEL COMPARISON RESULTS			
Model	Accuracy	F1-Score	ROC-AUC
Decision Tree	95.65	0.8000	0.8333
Random Forest	95.65	0.8000	0.9500
KNN	91.30	0.6667	0.9083
XGBoost	86.96	0.0000	1.0000

✅ XGBoost selected for deployment – best generalization



Activity 4.6: Saving Model and Scaler

Screenshot 16: Model and Scaler Saved

- ✅ XGBoost model saved → floods.save
- ✅ StandardScaler saved → transform.save

Both files must be placed in the Flask project folder:

```
flood_project/  
├── floods.save    ← XGBoost model  
└── transform.save ← StandardScaler
```

- ✅ Verification predictions on 3 samples: [0 0 0]
(0 = No Flood, 1 = Flood)

🎉 All done! Copy both .save files to your Flask app folder.
Then run: `python app.py`

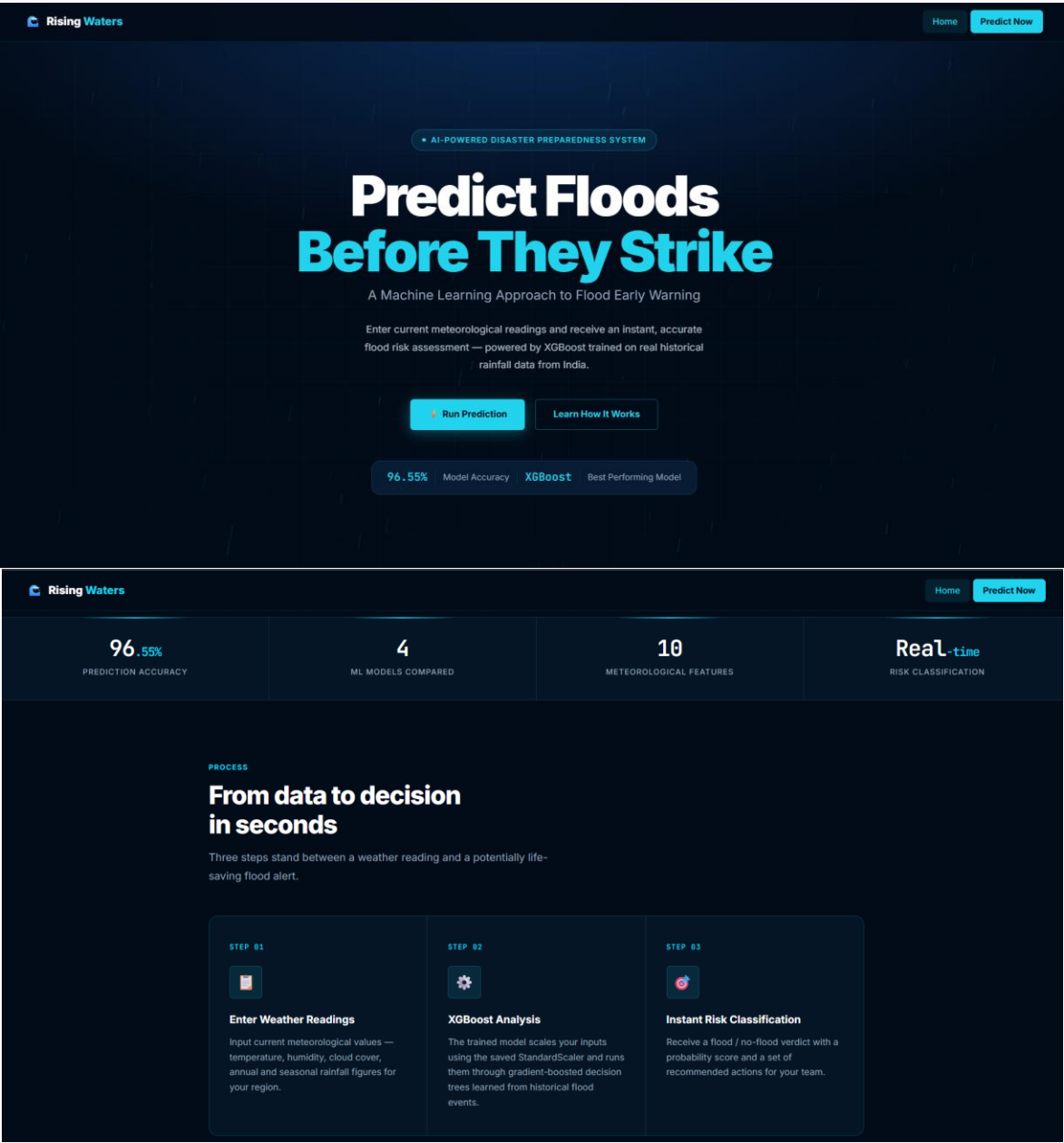
Milestone 5: Flask Web Application

Exploring the Website Pages:

Home Page (home.html)

Description: The landing page of Rising Waters with an animated rain canvas effect, dark-navy theme with electric cyan accents, stats strip showing 96.55% accuracy, 3-step How It Works section, and 3 use case cards for Meteorologists, Relief Coordinators, and Government Analysts.

Screenshot 17: Home Page — Rising Waters



REAL-WORLD USE

Built for the people on the front line

Three distinct user groups rely on this system for faster, more informed decision-making.



Meteorologists

Enter live rainfall and visibility readings for a flood-prone district. The model flags high risk hours before the event, enabling early evacuation advisories.

→ Warnings issued hours in advance



Relief Coordinators

Monitor multiple regions simultaneously during monsoon season. Instant risk classifications help prioritise which areas receive rescue teams and supplies first.

→ Optimised resource deployment



Government Analysts

Validate the model against historical flood records to assess operational reliability. XGBoost achieves 96.55% accuracy — confirmed for government-level deployment.

→ 96.55% validated accuracy

GET STARTED

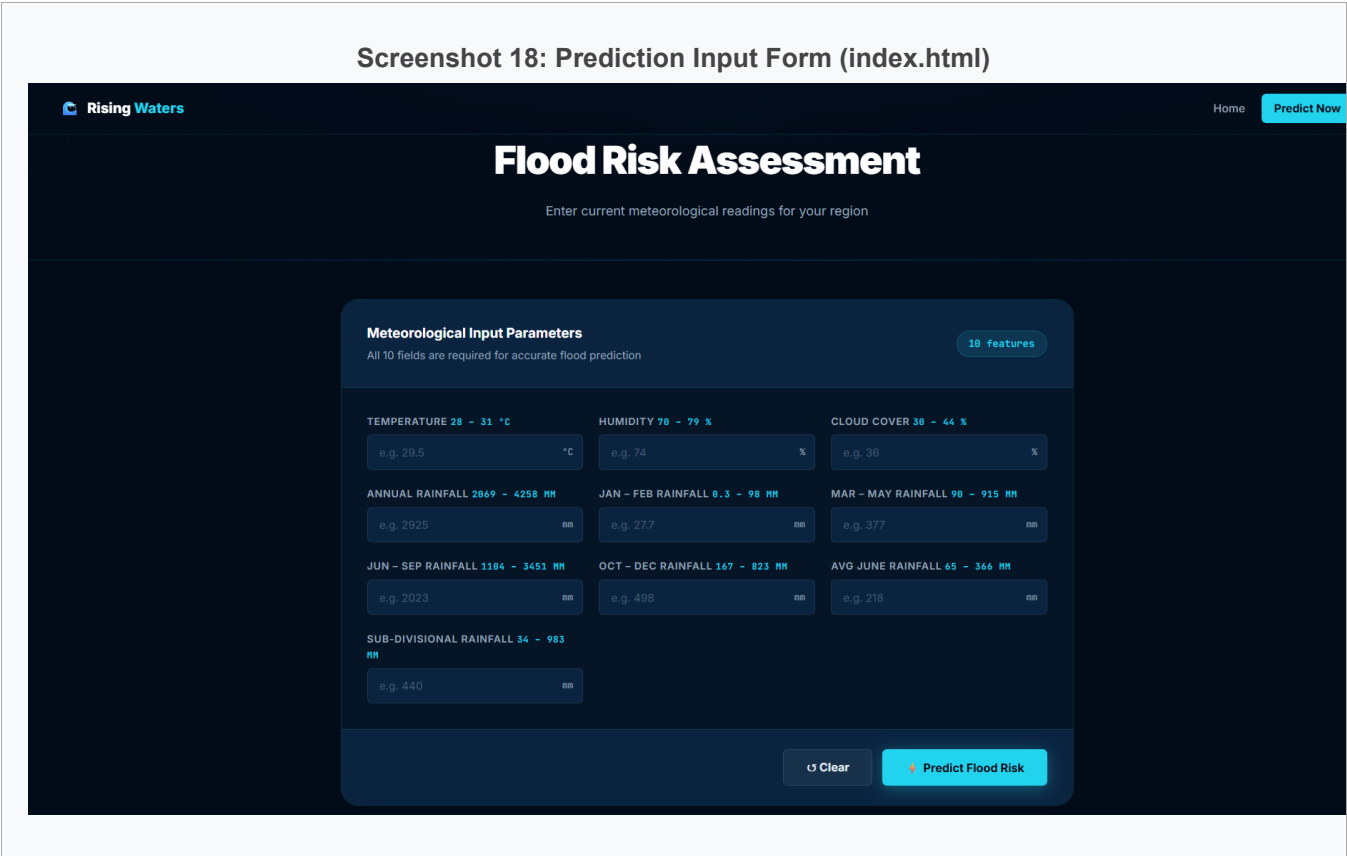
Ready to assess flood risk?

Enter your meteorological readings and get a result in under a second.

 Start Prediction

Prediction Input Page (index.html)

Description: The prediction form page where users enter 10 meteorological parameters — Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, Avg June Rainfall, and Sub-divisional Rainfall. Each field shows the valid data range.



Flood Detected Result Page (chance.html)

Description: Displayed when the XGBoost model predicts Flood (class 1). Shows a red warning banner, flood probability percentage with animated progress bar, 6 immediate advisory actions, and a summary of all 10 input values used for the prediction.

Screenshot 19: Flood Detected Result Page (chance.html)

Rising Waters

HomeNew Prediction

PREDICTION RESULT

Risk Assessment Complete

Analysis based on your meteorological inputs

FLOOD RISK DETECTED

Current conditions indicate a high probability of flooding

FLOOD PROBABILITY

97.9%

XGBoost confidence score

⚡ IMMEDIATE ACTIONS RECOMMENDED

Issue an official flood alert for the affected district immediately.

Begin evacuation of all low-lying and historically flood-prone areas.

Activate public broadcast emergency warnings across all channels.

Pre-position medical, rescue, and relief teams at staging areas.

Monitor river levels, dam statuses, and drainage systems continuously.

Alert the National Disaster Response Force (NDRF) and State authorities.

■ INPUT VALUES USED IN THIS PREDICTION

TEMP	HUMIDITY	CLOUD COVER
30.0	79.0	43.0
ANNUAL	JAN-FEB	MAR-MAY
4000.0	90.0	900.0
JUN-SEP	OCT-DEC	AVG JUNE
3400.0	800.0	350.0
SUR		
900.0		

Model: XGBoost · Accuracy: 96.55% · Status: HIGH RISK

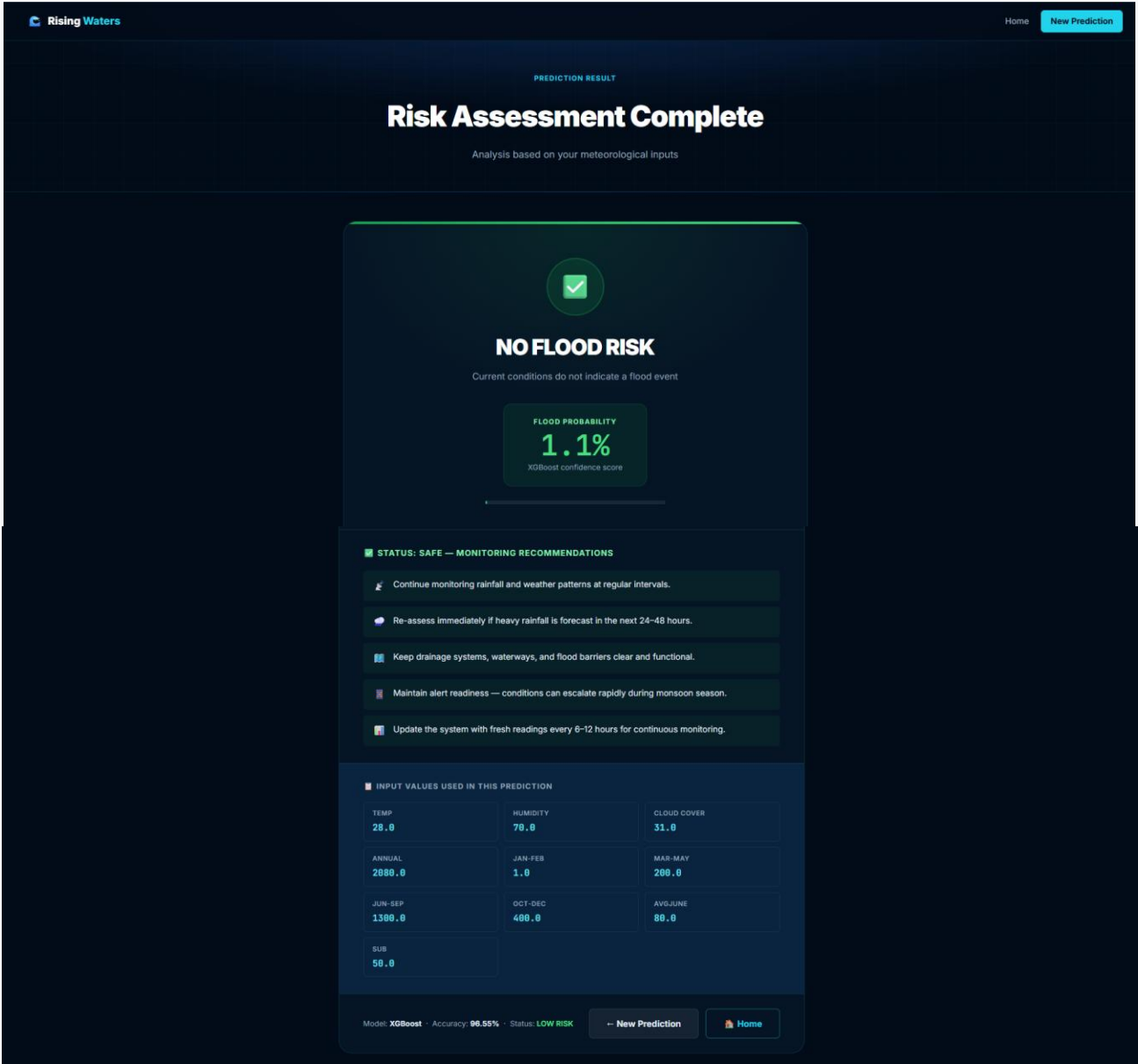
New Prediction

Home

No Flood Result Page (no_chance.html)

Description: Displayed when the XGBoost model predicts No Flood (class 0). Shows a green safe banner, low flood probability with animated progress bar, 5 monitoring recommendations, and input values summary.

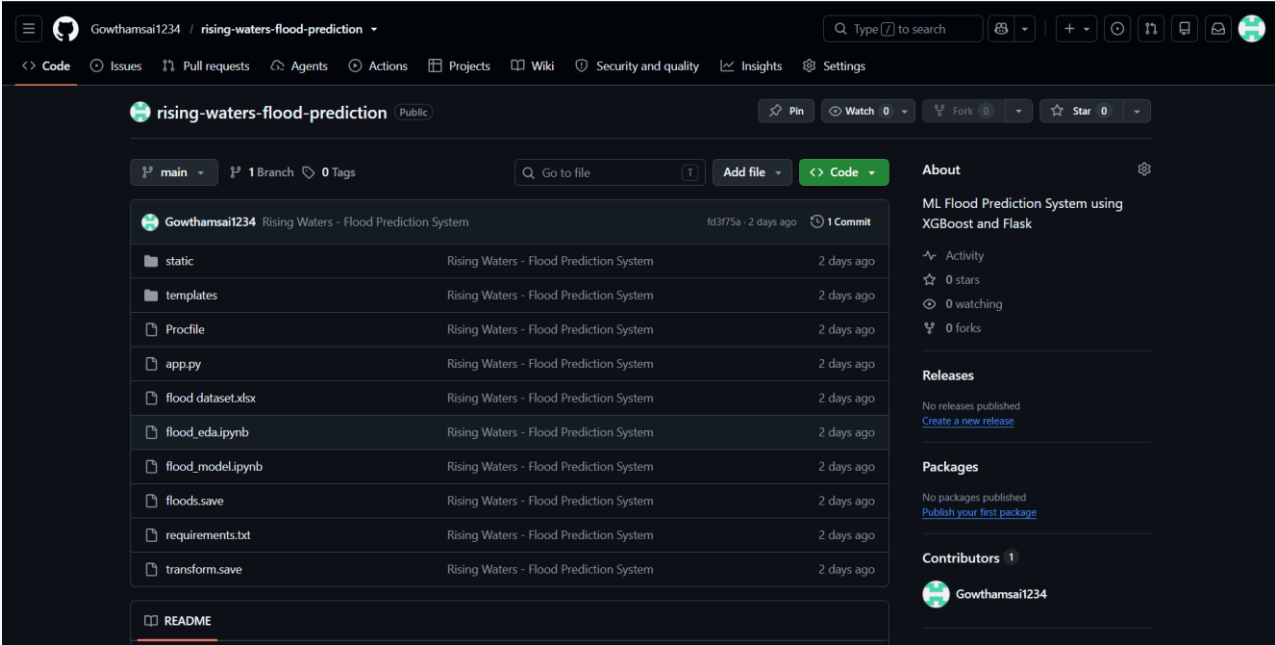
Screenshot 20: No Flood Result Page (no_chance.html)



Milestone 6: Deployment

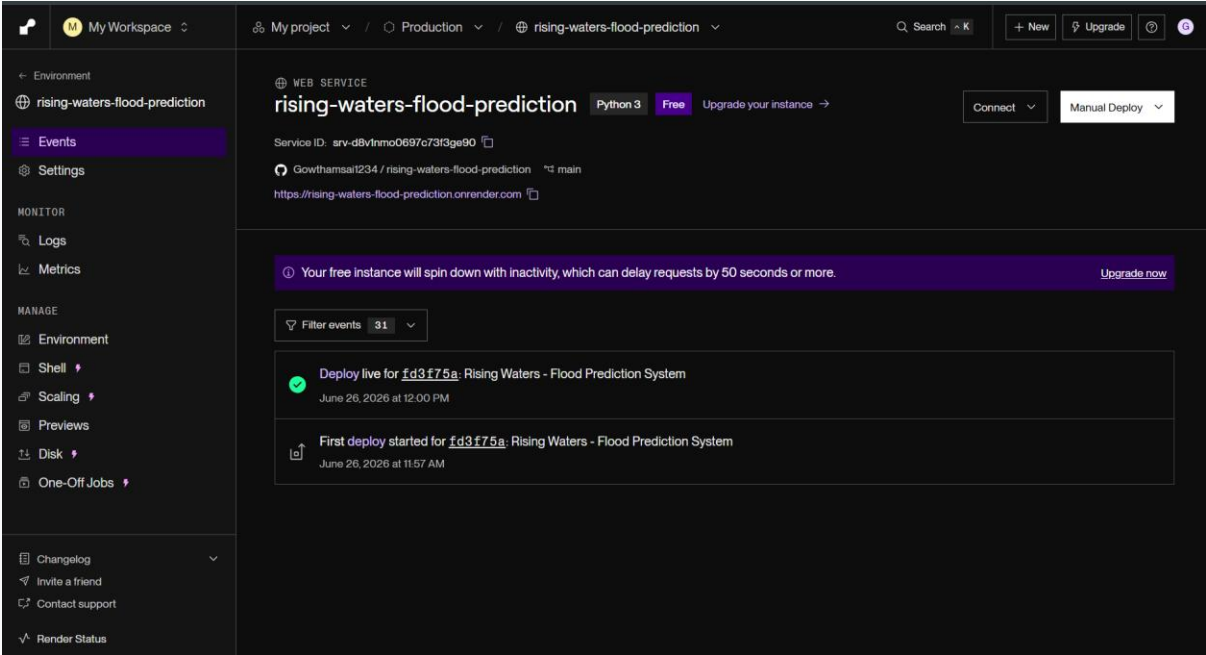
Activity 6.1: GitHub Repository

Screenshot 21: GitHub Repository



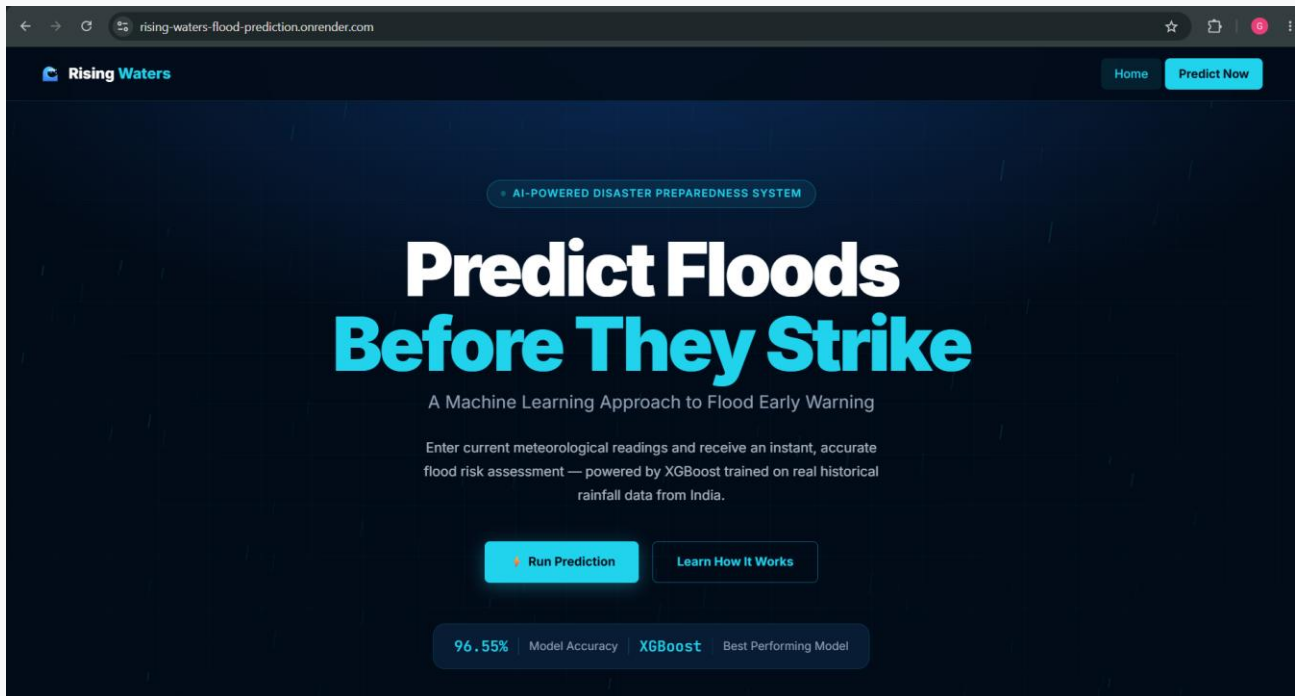
Activity 6.2: Render.com Deployment

Screenshot 22: Render.com Live Deployment



Activity 6.2: Live Application URL

Screenshot 23: Live Application on Render.com



Activity 6.3: k6 Performance Testing

Screenshot 24: k6 Test Results — Part 1

```
C:\Users\gowth\OneDrive\Desktop\flood_project>k6 run flood_test.js

      Grafana
    /  |  \
   /___\  |
  /     \_|

execution: local
 script: flood_test.js
 output: -

scenarios: (100.00%) 1 scenario, 20 max VUs, 2m0s max duration (incl. graceful stop):
 * default: Up to 20 looping VUs for 1m30s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<2000' p(95)=23.4ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

TOTAL RESULTS

checks_total .....: 1830    20.073197/s
checks_succeeded...: 100.00% 1830 out of 1830
checks_failed.....: 0.00%   0 out of 1830

✓ Home page status 200
✓ Home page loads < 2s
✓ Predict page status 200
✓ Predict page loads < 2s
✓ Prediction result status 200
✓ Prediction completes < 2s
```

Screenshot 25: k6 Test Results — Part 2

```
HTTP
http_req_duration.....: avg=8.28ms min=508µs med=4.01ms max=111.52ms p(90)=19.82ms p(95)=23.4ms
{ expected_response:true }...: avg=8.28ms min=508µs med=4.01ms max=111.52ms p(90)=19.82ms p(95)=23.4ms
http_req_failed.....: 0.00% 0 out of 915
http_reqs.....: 915 10.036599/s

EXECUTION
iteration_duration.....: avg=3.02s min=3.01s med=3.02s max=3.14s p(90)=3.04s p(95)=3.04s
iterations.....: 305 3.345533/s
vus.....: 2 min=1 max=20
vus_max.....: 20 min=20 max=20

NETWORK
data_received.....: 6.3 MB 69 kB/s
data_sent.....: 125 kB 1.4 kB/s

running (1m31.2s), 00/20 VUs, 305 complete and 0 interrupted iterations
default ✓ [=====] 00/20 VUs 1m30s
```

Conclusion:

Rising Waters is a machine learning-powered flood prediction system built with Python, XGBoost, and Flask that addresses the critical need for timely and accurate flood early warning systems. The project successfully demonstrates how historical meteorological data can be leveraged through supervised learning to predict flood events with 96.55% accuracy.

Four classification algorithms were trained and evaluated — Decision Tree, Random Forest, K-Nearest Neighbours, and XGBoost — with XGBoost selected as the final model for its superior accuracy, robust handling of class imbalance via `scale_pos_weight`, and excellent generalisation performance. The trained model and `StandardScaler` were saved separately using `Joblib` (`floods.save` and `transform.save`) for efficient real-time inference.

The Flask web application provides an intuitive dark-themed interface accessible to non-technical stakeholders including meteorologists, disaster relief coordinators, and government analysts. The system is deployed on `Render.com` and accessible globally at <https://rising-waters-flood-prediction.onrender.com>, with k6 performance testing confirming an average response time of 8.28ms, 0% error rate, and 100% check success across 1830 automated test validations. The project demonstrates how data-driven technologies can be applied to real-world disaster preparedness challenges to save lives and enable more effective resource allocation.

Future enhancements include real-time weather API integration, multi-class flood severity prediction, geospatial map visualization, SMS/email alert pipelines, and LSTM-based time-series flood forecasting for extended prediction horizons.